



Università
Ca'Foscari
Venezia

Linear Classifiers

FOUNDATIONS OF ARTIFICIAL INTELLIGENCE

Andrea Torsello

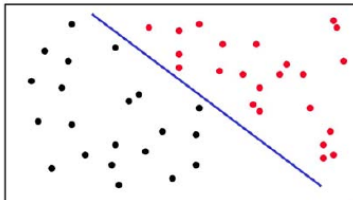
Dip. Sc. Ambientali, Informatica, Statistica
Università Ca'Foscari Venezia

Linear Separators

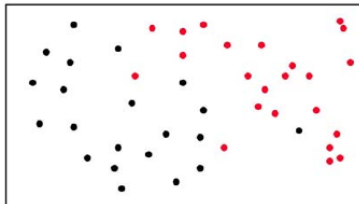
Linear separators are characterized by a single linear decision boundary in the space.

A **data set** that can be successfully split by a linear separator is called **linearly separable**

Linearly Separable



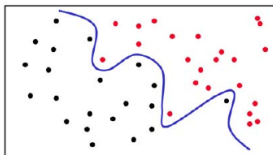
Not Linearly Separable



Non-Separable Data

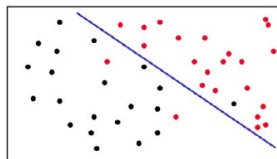
When faced with a non-linearly-separable data set, we have two options.

Not Linearly Separable



Use a more complex hypothesis class

Not Linearly Separable



keep the simple linear separator and accept some errors

- ▶ Classic Bias/Variance trade-off
- ▶ Which is more appropriate depends on the properties of the data, including the amount of noise.
- ▶ We can use cross-validation to make the choice if we don't have much understanding of the data.

Linear Hypothesis

How do we represent a particular hypothesis?

- ▶ equation of a linear separator.
- ▶ We will be illustrating in 2D but all the equations hold for an arbitrary number of dimensions.

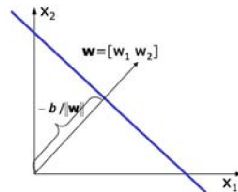
The equation of a linear separator in an n -dimensional feature space is a linear equation determined by $n + 1$ values:

- ▶ n -dimensional coefficient vector w (weights)
- ▶ A scalar value b (bias)

Linear Hypothesis

w is perpendicular to the linear separator
(normal vector)

b is proportional to the perpendicular distance from the origin to the linear separator.



- ▶ The constant of proportionality is the negative of the magnitude of the normal vector.
- ▶ If we set w of unit length, b is the perpendicular distance

w and b are to be **learned**

x is a point in the **feature space**

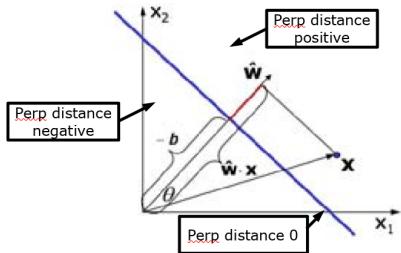
Useful trick: add a new dimension to x and w so that $x_{n+1} = 1$
and $w_{n+1} = b$

This way the equation reduces to $x \cdot w = 0$

Hyperplane Geometry

$$x \cdot w$$

Signed perpendicular distance
of x to the hyperplane



Exploit the sign of this distance to define a linear classifier

- use the sign of the distance, either +1 or -1 as the labels (that is the values of the y_i)

$$h(x) = \text{sign}(x \cdot w)$$

Margin

The **margin** is the product of the signed distance and the desired label y_i

$$\gamma_x = (x \cdot w)y_i$$

- ▶ If they agree (correct classification) the margin is positive
- ▶ If they disagree (incorrect classification) the margin is negative

the margin is in the units given by 1 over norm of w ,

the units given by the w itself.

The Perceptron Algorithm (Rosenblatt, 1956)

Perceptron

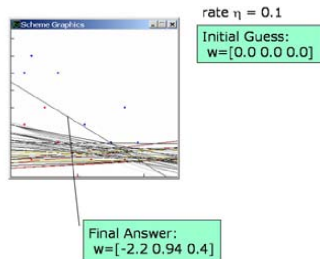
```
def perceptron(x, y, initial_w, rate):  
    w = initial_w  
    all_correct = False  
    n = len(x)  
    while (not all_correct):  
        all_correct = True  
        for i in range(n):  
            gamma = (w @ x[i]) * y[i]  
            if gamma < 0:  
                all_correct = False  
                w += rate * y[i] * x[i]  
    return w
```


The Perceptron Algorithm (Rosenblatt, 1956)

Each change of w increases the margin on a particular point.

However, changes interfere with each other

- ▶ one pass through the will not be enough



The algorithm is guaranteed to converge to a separating hyperplane for linearly separable data.

Motivation

Why is it reasonable to increase weights by $y_i x_i$?

Maximize sum of margins of misclassified points

$$f(w) = \sum_{i \in \mathcal{M}} y_i w \cdot x_i$$

Use a gradient method

$$w^{(t+1)} = w^{(t)} + \eta \nabla_{w^{(t)}} f(w^{(t)})$$

with

$$\nabla_w f(w) = \sum_{i \in \mathcal{M}} y_i x_i$$

Off-line training: Compute gradient as sum over all training points

On-line training: update on each new misclassification.

Approximate gradient by a single term

Dual form of the Perceptron

The perceptron algorithm starts with an initial guess for the weights

Then adds in scaled versions of the misclassified training points to get the final weights

If we sum up the coordinates of each point, scaled by how many times each is misclassified and by the rate constant we get the total change in the weight vector.

$$w^{(t)} - w^{(0)} = \eta \sum_{i=1}^n \alpha_i y_i x_i$$

where α_i is the number of times x_i was misclassified

Dual form of the Perceptron

Since the rate does not change the separator, set $\eta = 1$

Assuming $w^{(0)} = 0$, we have $w^{(t)} = \sum_{i=1}^n \alpha_i y_i x_i$

We can rewrite the classifier as

$$h(x) = \text{sign} \left(\left(\sum_{i=1}^n \alpha_i y_i x_i \right) \cdot x \right)$$

If data is not linearly separable, then the α_i grow without bound

Dual Perceptron

Dual Perceptron

```
def dual_perceptron(x, y):  
    all_correct = False  
    n = len(x)  
    alpha = np.zeros(n)  
    while (not all_correct):  
        all_correct = True  
        for i in range(n):  
            w = (y*alpha)@x  
            gamma = (w@x[i])*y[i]  
            if gamma < 0:  
                all_correct = False  
                alpha[i] += 1  
    return w = (y*alpha)@x
```

γ is the margin, which is the distance between the decision boundary (hyperplane) and the closest data points.

Convergence of the Perceptron

Theorem: The Perceptron converges in a finite number of steps

Assumptions: $R = \max ||x_i||$

- ▶ the (training) points have bounded Euclidean norms, i.e., $||x_i|| \leq R$ for all i and some finite R . This is clearly the case for any finite set. R is the maximum length (norm) of the input vectors.
- ▶ There exists a linear separator w^* which correctly classifies all our data with margin at least $\gamma > 0$

We need to show two results:

1. $w^* \cdot w^{(t)}$ increases at least linearly with each update
2. $||w^{(t)}||^2$ increases at most linearly with each update

Then, since $w^* \cdot w^{(t)} \leq ||w^*||^2 ||w^{(t)}||^2$, we can make only a finite number of steps.



The linear separator w is the weight vector that defines the decision boundary (or hyperplane) in the feature space for linearly separable data.

Convergence of the Perceptron

Cauchy-Schwarz inequality: The dot product between (the correct weight) and (the current weight) is bounded by the product of their magnitudes (squared). In simple terms, this means that the inner product of these two vectors cannot grow faster than their magnitudes multiplied together.

1. Assuming that at time t we misclassify point i , we have

$$w^* \cdot w^{(t+1)} = w^* \cdot w^{(t)} + y_i w^* \cdot x_i \geq w^* \cdot w^{(t)} + \gamma$$

Hence, after k updates we have

$$w^* \cdot w^{(k)} \geq k\gamma$$

It means that with each update, the dot product $w(t)$ increases by a certain amount, and the increase is proportional to the number of updates, ensuring that $w(t)$ is getting closer to the optimal weight vector w^* over time.

w^* is the optimal solution, that separate the data correctly
 $w(t)$ is the current weight of t time.

Convergence of the Perceptron

2. Our second claim follows simply from the fact that updates are made only on mistakes:

$$\begin{aligned}\|w^{(k)}\|^2 &= \|w^{(k-1)} + y_i x_i\|^2 \\ &= \|w^{(k-1)}\|^2 + 2y_i w^{(k-1)} \cdot x_i + \|x_i\|^2 \\ &\leq \|w^{(k-1)}\|^2 + \|x_i\|^2 \\ &\leq \|w^{(k-1)}\|^2 + R^2\end{aligned}\tag{1}$$

thus,

$$\|w^{(k)}\|^2 \leq kR^2$$

We can combine the two results since

$$1 \geq \cos(w^*, w^{(k)}) = \frac{w^* \cdot w^{(k)}}{\|w^*\| \|w^{(k)}\|} \geq \frac{k\gamma}{\|w^*\| \sqrt{k}R}$$

or

$$k \leq \frac{R^2 \|w^*\|^2}{\gamma^2}$$

$\|w(t)\| \leq kR^2$ It means that the squared magnitude of the weight vector $w(t)$ grows at a rate proportional to the number of updates made during the perceptron algorithm, ensuring that it doesn't grow too quickly.

Generalization

Let's assume then that all the data-points we could possibly encounter satisfy the same two assumptions.

1. $\|x_i\| \leq R$
2. $y_i w^* \cdot x_i \geq \gamma$

That is there is an (unknown) linear classifier that works on all instances of the data

Let's now imagine getting the data-points one by one and performing only a single update per point, if misclassified, and move on.

How many mistakes are we now going to make in this infinite arbitrary sequence of images and labels, subject only to the two assumptions?

Generalization

How many mistakes are we now going to make?

The same number

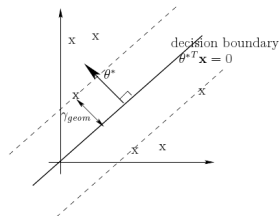
$$k \leq \frac{R^2 \|w^*\|^2}{\gamma^2}$$

Once we have made this many mistakes we would classify all the new points correctly!

Geometric Margin

Does $\frac{\|w^*\|^2}{\gamma^2}$ relate to how difficult the problem is?

Indeed! It is the smallest distance from any example to the decision boundary



We will call the quantity $\gamma_{\text{geom}} = \frac{\gamma}{\|w^*\|}$ the **geometric margin**

The Perceptron is a simple on-line algorithm that finds a linear classifier **when data is linearly separable** by a **large geometric margin**

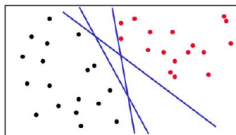
Can we find such a large margin classifier **directly**?

- Yes, we can. The classifier is known as the Support Vector Machine or SVM

Large Margin Classifiers

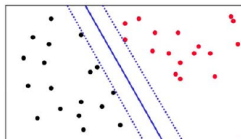
In general, there can be many separators for a data set.

There is no easy way to characterize which particular separator the perceptron algorithm will choose



Is there any reason to prefer one separator over the others?

One natural choice is to pick one that has the largest geometric margin



The line that will not be affected by changing or adding any data points only affected by the data points on it. It is the most stable one in the sense that it generalizes better.

Maximizing the Margin

$\gamma_i = y_i w \cdot x_i$ is proportional to the perpendicular distance of x_i to the hyperplane

The geometric margin is $\frac{\gamma_i}{\|w\|}$

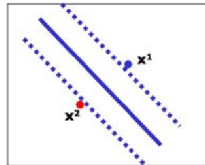
- Scaling w changes the perpendicular distance but not the geometric distance

Set margin to closest points to 1 and **maximize the geometric margin**

it doesn't change the classifier but change the value of the margin.

$$\begin{aligned} +1(w \cdot x_1) &= 1 \\ -1(w \cdot x_2) &= 1 \end{aligned}$$

every point has margin at least one, that is we can set w in such a way that the min margin is at least one.



Combining these we have

once we set the scale of w such that the margin is 1, the geometric margin is 1 over the norm of w .

$$\frac{w}{\|w\|} \cdot (x_1 - x_2) = \frac{2}{\|w\|}$$

it is a geometric perpendicular distance, that's why 2 over norm of w

The geometric margin is always scale-independent and reflects the true distance

Support Vector Machines

Choose w to maximize $2/\|w\|$, or equivalently, to **minimize** $\|w\|^2$

$$\begin{array}{ll} \min_w & \frac{1}{2} \|w\|^2 \\ \text{subject to} & \forall i \in \{1, \dots, n\}. y_i(w \cdot x_i) - 1 \geq 0 \end{array}$$

The objective is to find the w that optimize the obj. function while satisfying the margin constraint.

The problem is constrained, requiring every **margin** to be greater than or equal to 1

Turn it into an unconstrained one with Lagrange multipliers

$(\alpha = (\alpha_1, \dots, \alpha_n), \alpha_i \geq 0)$ if $\alpha_i > 0$, the data point is a support vector

$$\min_w \left(\frac{1}{2} \|w\|^2 - \sum_{i=1}^n \alpha_i [y_i(w \cdot x_i) - 1] \right) \quad \text{convex problem, hence has single minima}$$

α_i is the Lagrange multiplier, which is a scalar value that represents how much the constraint affects the objective function. α_i determines the influence of the corresponding data point on the margin.

Support Vector Machines

We have an expression that also involves parameters α

If we knew what the values of alpha should be, we could just fix them, minimize with respect to w and be done

The big problem is that we don't know what the α 's are supposed to be

We find the optimal set of parameters $w^*(\alpha)$ as a function of α , that minimize the Lagrangian

$$w^*(\alpha) = \sum_{i=1}^n \alpha_i y_i x_i$$

In SVMs, the multipliers α_i correspond to the constraints on each data point and represent the "weight" of the constraint in defining the optimal hyperplane. The function $w(\alpha)$ represents the optimal solution for w in terms of the dual variable α .

Support Vector Machines (Dual Form)

Substituting $w * (\alpha)$ into the Lagrangian, yields the dual form

$$\begin{aligned} \underset{\alpha}{\text{maximize}} \quad & \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) \\ \text{subject to} \quad & \forall i \in \{1, \dots, n\}. \alpha_i \geq 0, \sum_{i=1}^n \alpha_i y_i = 0 \end{aligned}$$

Now we can optimize with respect to α

- ▶ The problem is **convex**
- ▶ If $\alpha_i = 0$ the constraint is satisfied with no distortion
- ▶ If $\alpha_i > 0$ the constraint is satisfied with equality
In this case x_i is a **support vector**

Support vectors are a small fraction of the dataset and completely determine the classifier

SVM Classifier

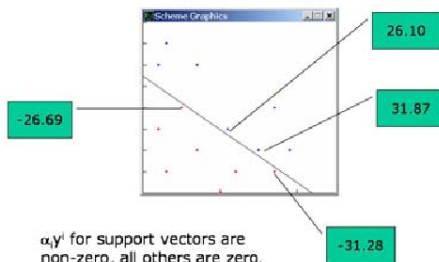
Given unknown vector u , predict class $(-1,1)$ as follows:

$$h(u) = \text{sign} \left(\sum_{i=1}^k \alpha_i y_i (x_i \cdot u) \right)$$

The sum is over k support vectors

The system has a unique optimal solution

Can be found using quadratic programming or gradient ascent



Recap

Max margin lowers hypothesis variance More stable prediction

The optimal classifier is defined uniquely (no local maxima)

Learning Polynomial in number of data and dimensionality

The classifier depends only on the support vectors

Classification linear in number of points in support vector

Learning depends only on dot product of sample pairs

Classification depends only on dot product of unknown with samples in the support vector

Still requires data linearly separable

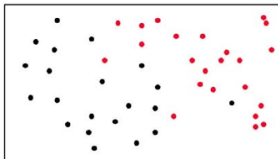
- ▶ Allow misclassification
- ▶ Allow non-linear boundaries

Non-Separable case

We have only been talking about the linearly separable case.

What happens when we have a “nearly separable” problem?

Not Linearly Separable



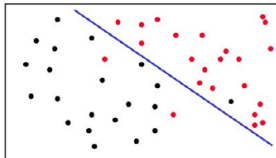
We can see that if point i stubbornly resists being classified, then the value of α_i will continue to grow without bounds.

- ▶ The α 's in the dual perceptron are analogous to the values of the Lagrange multipliers in the SVM.
- ▶ In both cases, the separator is defined as a linear combination of the input points, with the alphas being the weights.

Non-Separable case

We would like to find a separator that allows misclassification

Not Linearly Separable



Lighten the margin constraint

Soft Margin

SVM classifiers are learned by optimizing the following

$$\begin{array}{ll} \min_w & \frac{1}{2} \|w\|^2 \\ \text{subject to} & y_i(w \cdot x_i) - 1 \geq 0 \quad \forall i \end{array}$$

The enforcement of the margin is due to the hard constraint

We can render the condition less strict by adding slack variables to allow for deviation and penalizing deviation

Soft Margin

This results in the following formulation

$$\begin{aligned} \min_w \quad & \frac{1}{2} \|w\|^2 + C \sum_i \zeta_i \\ \text{subject to} \quad & y_i(w \cdot x_i) \geq 1 - \zeta_i \quad \forall i \\ & \zeta_i \geq 0 \quad \forall i \end{aligned}$$

Where C is a balance term controlling how strictly the margin must be enforced

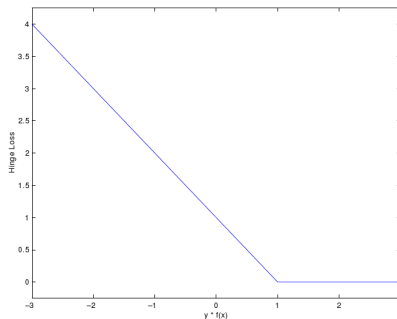
Hinge Loss

The optimal slack variable satisfies

$$\zeta_i = \max(1 - y_i(w \cdot x_i), 0)$$

With this SVM can be seen as minimizing the **hinge loss** function

$$V(f(x, y)) = \max(1 - y(w \cdot x), 0)$$



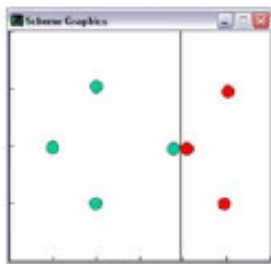
Soft Margin in the Dual

In the dual form the hinge loss turns into an upper limit for the dual variables α_i

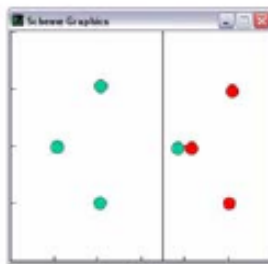
$$\begin{aligned} & \underset{\alpha}{\text{maximize}} && \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) \\ & \text{subject to} && 0 \leq \alpha_i \leq C \quad \forall i \\ & && \sum_{i=1}^n \alpha_i y_i = 0 \end{aligned}$$

Meaning of Parameter C

- ▶ By choosing a large value of C , we will work very hard at correctly classifying all the points
- ▶ A low value of C will allow us to give up more easily on many of the points so as to achieve a better margin.

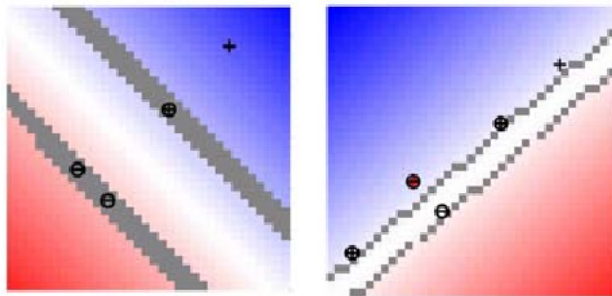


$C=10$



$C=1$

Example classification

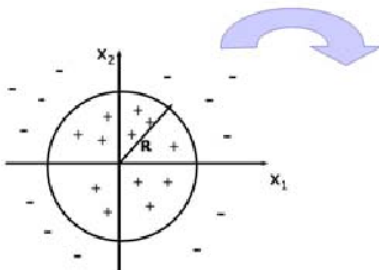


Make it Non-Linear

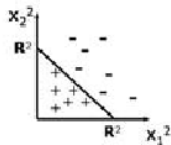
Even if we provide a mechanism for ignoring noise points, aren't we really limited by a linear classifier?

► YES!

In many cases, if we transform the feature values in a non-linear way, we can transform a problem that was not linearly separable into one that is.



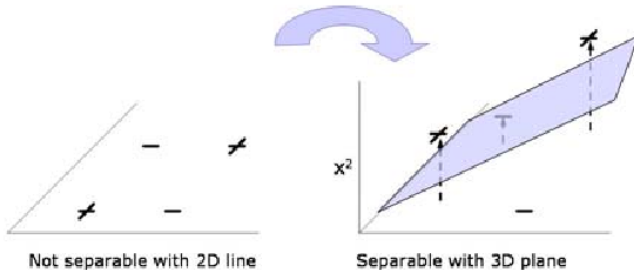
not linearly
separable



linearly separable using
squared value of features.

Make it Non-Linear

Similarly, when training samples are not separable in the original space they may be separable if you perform a transformation into a higher dimensional space



Non-Linear Embedding

To get to the new feature space, use the function $\phi(x)$

- ▶ The transformation can be to a higher-dimensional space and can be non-linear

Recall that **SVM only use dot products of the data**

- ▶ To optimize (train) the classifier, you need $\phi(x_i) \cdot \phi(x_j)$
- ▶ To run classifier on a new point u you need $\phi(x_i) \cdot \phi(u)$

You need a way to compute dot products in the transformed space as a function of vectors in the original space

- ▶ Suppose you have a **non-linear** function K such that

$$\phi(x_i) \cdot \phi(x_j) = K(x_i, x_j)$$

- ▶ No need to compute the high-dimensional mapping $\phi(x)$!

Positive Definiteness

A function $K : X \times X \rightarrow \mathbb{R}$ is said to be **positive definite** is, for any n and any choice of the set $\{x_1, \dots, x_n\} \subseteq X$, the matrix

$$M = \begin{pmatrix} K(x_1, x_1) & \cdots & K(x_1, x_n) \\ \vdots & \ddots & \vdots \\ K(x_n, x_1) & \cdots & K(x_n, x_n) \end{pmatrix}$$

is positive definite

Mercer's Kernels

Theorem (Mercer's Representer Theorem)

Let $K : X \times X \rightarrow \mathbb{R}$ be a positive-definite function, then there exist a (possibly infinite-dimensional) vector space Y and a function $\phi : X \rightarrow Y$ such that

$$K(x_1, x_2) = \phi(x_1) \cdot \phi(x_2)$$

This means that you can substitute dot products with any positive-definite function K (called **kernel**) and you have an implicit non-linear mapping to a high-dimensional space

If you chose your kernel properly, your decision boundary bends to fit the data

Kernels

There are various choices for kernels

Linear kernel $K(x_i, x_j) = x_i \cdot x_j$

Polynomial kernel $K(x_i, x_j) = (1 + x_i \cdot x_j)^n$

Radial Basis Function $K(x_i, x_j) = \exp\left(-\frac{1}{2} \frac{\|x_i - x_j\|^2}{\sigma^2}\right)$

Example

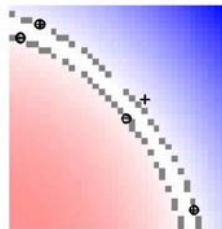
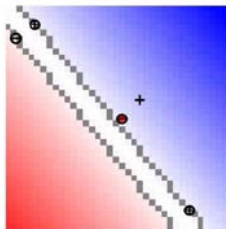
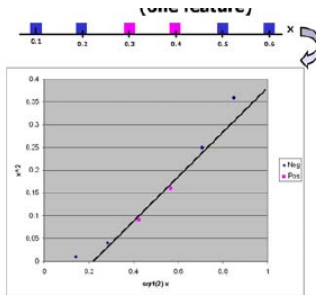
Polynomial kernel $n=2$, equivalent to mapping

$$\phi(x) = (x_1^2, x_2^2, \sqrt{2}x_1x_2, \sqrt{2}x_1, \sqrt{2}x_2, 1)^T$$

We can verify that

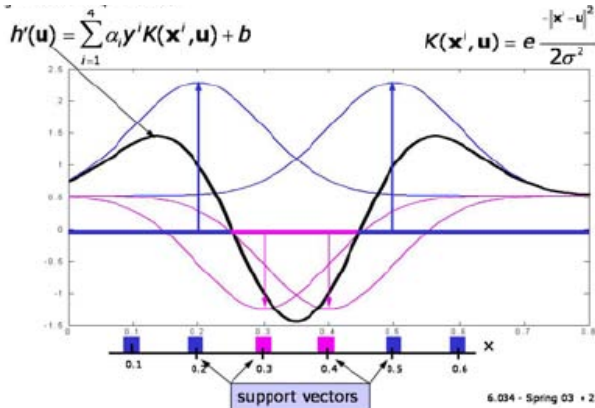
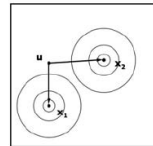
$$\begin{aligned}\phi(x) \cdot \phi(z) &= x_1^2 z_1^2 + x_2^2 z_2^2 + 2x_1 x_2 z_1 z_2 + 2x_1 z_1 + 2x_2 z_2 + 1 \\ &= (1 + x_1 z_1 + x_2 z_2)^2 \\ &= (1 + x \cdot z)^2 \\ &= K(x, z)\end{aligned}$$

Example: Polynomial Kernel $n=2$

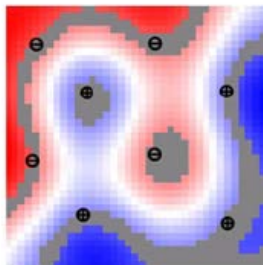
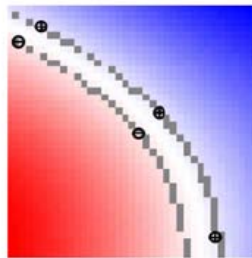
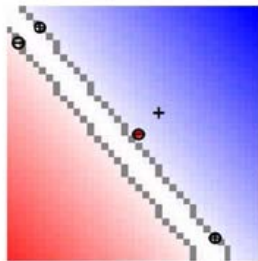


Radial Basis Kernel

Classifier based on sum of Gaussian bumps centered on support vectors



Example: RBK



Generalization

Does mapping to a very high-dimensional space lead to over-fitting?

- ▶ Generally no, thanks to the fact that only the support vectors determine the decision surface

The expected leave-one-out cross-validation error depends on the support vectors, not on the dimensionality of the feature space

$$\text{expected CV error} \leq \frac{\text{expected num of support vectors}}{\text{number of training samples}}$$

If most points are support vectors it is a sign of over-fitting